

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №4
по дисциплине: Операционные системы
по теме: Разработка драйвера для ОС Linux (Ubuntu)

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: доц. Островский Алексей Мичеславович,
асс. Четвертухин Виктор Романович

Белгород 2024

Цель работы: Изучить основы разработки драйверов для ядра Linux с использованием языка программирования C, включая настройку окружения, создание драйвера и его тестирование.

Цель работы обуславливает постановку и решение следующих задач:

1. Настроить окружение для разработки драйвера на языке C для ОС Linux (Ubuntu).
2. Разработать и протестировать демо-драйвер `my_sum` на языке C.
 - 2.1. Освоить основные принципы программирования драйвера.
 - 2.2. Реализовать работу с мьютексами в драйвере.
3. Выполнить индивидуальное задание, закрепляющее на практике полученные знания (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления).
 - 3.1. Добавить логирование операций с использованием встроенных средств ядра Linux.
 - 3.2. Создать структуру для управления ресурсами драйвера, включая выделение и освобождение памяти.
 - 3.3. Протестировать работу разработанного драйвера на производительность при высоком числе одновременных запросов.
4. Составить отчет, включающий код разработанного драйвера, результаты тестирования и выводы.

Создал backups виртуальной машины

```
wsl --export Ubuntu-24.04 C:\backups\ubuntu-24.04.tar
```

Ход выполнения лабораторной работы

1. Настройка окружения

Установите необходимые пакеты:

```
sudo apt update && sudo apt install build-essential gcc make
```

2. Создание драйвера для подсчета суммы чисел.

```
mkdir ~/my_sum_driver
```

```
cd ~/my_sum_driver
```

Файл ~/my_sum/my_sum.c:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/mutex.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("A.M.Ostrowski");
MODULE_DESCRIPTION("A driver for summing numbers via /dev/my_sum");
MODULE_VERSION("1.0");

#define DEVICE_NAME "my_sum"
#define BUFFER_SIZE 256

static int major_number;
static int current_sum = 0; // Сумма
static char buffer[BUFFER_SIZE];
static size_t buffer_size = 0;
static struct mutex sum_mutex; // Мьютекс

static int device_open(struct inode *inodep, struct file *filep) {
    printk(KERN_INFO "my_sum: Device opened\n");
    return 0;
}

static int device_release(struct inode *inodep, struct file *filep) {
    printk(KERN_INFO "my_sum: Device closed\n");
    return 0;
}
```

```

}

static ssize_t device_read(struct file *filep, char *user_buffer, size_t len,
loff_t *offset) {
    int ret;
    if (*offset > 0)
        return 0;
    mutex_lock(&sum_mutex);

    buffer_size = snprintf(buffer, BUFFER_SIZE, "%d\n", current_sum);
    mutex_unlock(&sum_mutex);
    ret = copy_to_user(user_buffer, buffer, buffer_size);
    if (ret == 0) {
        printk(KERN_INFO "my_sum: Sent sum %d to user\n", current_sum);
        *offset += buffer_size;
        return buffer_size;
    } else {
        printk(KERN_ERR "my_sum: Failed to send data to user\n");
        return -EFAULT;
    }
}

static ssize_t device_write(struct file *filep, const char *user_buffer, size_t
len, loff_t *offset) {
    char input[BUFFER_SIZE] = {0};
    int number;
    if (len >= BUFFER_SIZE) {
        printk(KERN_ERR "my_sum: Input is too long\n");
        return -EINVAL;
    }
    if (copy_from_user(input, user_buffer, len)) {
        printk(KERN_ERR "my_sum: Failed to copy data from user\n");
        return -EFAULT;
    }
    input[len] = '\0';
    if (kstrtoint(input, 10, &number) != 0) {
        printk(KERN_ERR "my_sum: Invalid number format\n");
        return -EINVAL;
    }

    mutex_lock(&sum_mutex);
    if (!number)
        current_sum = 0;
    else
        current_sum += number;

    printk(KERN_INFO "my_sum: Added %d, new value is %d\n", number, current_sum);
    mutex_unlock(&sum_mutex);

    return len;
}

```

```

static struct file_operations fops = {
    .open = device_open,
    .read = device_read,
    .write = device_write,
    .release = device_release,
};

static int __init my_sum_init(void) {
    major_number = register_chrdev(0, DEVICE_NAME, &fops);

    if (major_number < 0) {
        printk(KERN_ERR "my_sum: Failed to register device\n");
        return major_number;
    }

    mutex_init(&sum_mutex);
    printk(KERN_INFO "my_sum: Registered with major number %d\n", major_number);
    return 0;
}

static void __exit my_sum_exit(void) {
    mutex_destroy(&sum_mutex);
    unregister_chrdev(major_number, DEVICE_NAME);
    printk(KERN_INFO "my_sum: Device unregistered\n");
}

module_init(my_sum_init);
module_exit(my_sum_exit);

```

3. Настройка Makefile:

```

obj-m += my_sum.o
KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
    make -C $(KDIR) M=$(PWD) modules
clean:
    make -C $(KDIR) M=$(PWD) clean

```

4. Собираем драйвер (из корневой директории исходного кода ядра Linux):

```
make clean
```

```
make -j$(nproc)
```

5. Загружаем модуль:

```
sudo insmod my_sum.ko
```

2. Создание драйвера индивидуальное задание

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/slab.h>
#include <linux/mutex.h>

#define DEVICE "morse_device"
#define MAX_BUF 1024

static int dev_major;
static char *buffer;
static struct mutex buffer_lock;

static const char *alphabet_morse[26] = {
    ".-", "-...", "-.-.", "-..", ".", "..-", "--.", "...", "..", ".---",
    "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.", "...", "-",
    "...-", "...-", ".--", "-..-", "-.-", "-...-"
};

static const char *digit_morse[10] = {
    "-----", ".-----", "..-----", "...-----", "....-",
    ".....", "-.....", "--.....", "---.....", "-----"
};

static void convert_to_morse(const char *src, char *dst) {
    int append_space = 0;

    while (*src) {
        if (append_space) {
            strcat(dst, " ");
        }

        if (*src >= 'a' && *src <= 'z') {
            strcat(dst, alphabet_morse[*src - 'a']);
            append_space = 1;
        } else if (*src >= 'A' && *src <= 'Z') {
            strcat(dst, alphabet_morse[*src - 'A']);
            append_space = 1;
        } else if (*src >= '0' && *src <= '9') {
            strcat(dst, digit_morse[*src - '0']);
            append_space = 1;
        } else if (*src == ' ') {
            strcat(dst, " ");
        }

        src++;
    }
}
```

```

static ssize_t morse_write(struct file *filp, const char __user *user_buf, size_t
len, loff_t *off) {
    if (len >= MAX_BUF) {
        pr_err("%s: Input exceeds buffer size\n", DEVICE);
        return -EINVAL;
    }

    mutex_lock(&buffer_lock);

    if (copy_from_user(buffer, user_buf, len)) {
        mutex_unlock(&buffer_lock);
        return -EFAULT;
    }

    buffer[len] = '\0';
    pr_info("%s: Input received: %s\n", DEVICE, buffer);

    mutex_unlock(&buffer_lock);
    return len;
}

static ssize_t morse_read(struct file *filp, char __user *user_buf, size_t len,
loff_t *off) {
    char *morse_buf;
    int read_len;

    if (*off > 0)
        return 0;

    morse_buf = kzalloc(MAX_BUF, GFP_KERNEL);
    if (!morse_buf)
        return -ENOMEM;

    mutex_lock(&buffer_lock);
    convert_to_morse(buffer, morse_buf);
    mutex_unlock(&buffer_lock);

    read_len = simple_read_from_buffer(user_buf, len, off, morse_buf,
strlen(morse_buf));

    kfree(morse_buf);
    return read_len;
}

static int morse_open(struct inode *inode, struct file *filp) {
    return 0;
}

static int morse_close(struct inode *inode, struct file *filp) {
    return 0;
}

```

```

}

static const struct file_operations fops = {
    .owner = THIS_MODULE,
    .write = morse_write,
    .read = morse_read,
    .open = morse_open,
    .release = morse_close
};

static int __init morse_init(void) {
    dev_major = register_chrdev(0, DEVICE, &fops);
    if (dev_major < 0) {
        pr_err("%s: Device registration failed\n", DEVICE);
        return dev_major;
    }

    buffer = kzalloc(MAX_BUF, GFP_KERNEL);
    if (!buffer) {
        unregister_chrdev(dev_major, DEVICE);
        return -ENOMEM;
    }

    mutex_init(&buffer_lock);
    pr_info("%s: Registered with major %d\n", DEVICE, dev_major);
    return 0;
}

static void __exit morse_exit(void) {
    kfree(buffer);
    unregister_chrdev(dev_major, DEVICE);
    pr_info("%s: Unregistered\n", DEVICE);
}

module_init(morse_init);
module_exit(morse_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Morse Code Converter Kernel Module");

```

3. Настройка Makefile:

```

obj-m += morse_code.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

```

4. Собираем драйвер:

```
make clean
```

```
make
```

5. Загружаем модуль:

```
sudo insmod morse_code.ko
```

Проверяем, загружен ли драйвер:

```
lsmod | grep morse_code
```

```
vboxuser@ubuntu:~/Desktop/src/os4$ sudo insmod morse_code.ko
vboxuser@ubuntu:~/Desktop/src/os4$ lsmod | grep morse_code
morse_code                12288  0
```

Проверяем лог:

```
sudo dmesg | grep "morse_code "
```

```
vboxuser@ubuntu:~/Desktop/src/os4$ sudo dmesg | grep "morse_code"
[ 7573.259960] morse_code: loading out-of-tree module taints kernel.
[ 7573.259980] morse_code: module verification failed: signature and/or required key missing - tainting kernel
[ 7573.266671] morse_code: device registered with major 240
vboxuser@ubuntu:~/Desktop/src/os4$
```

Создаём файл-устройство:

```
sudo mknod /dev/morse_code c <major> 0
```

Где <major> — номер, указанный в выводе dmesg.

Проверяем результат:

```
cat /dev/my_sum
```

```
make -C /lib/modules/6.8.0-49-generic/build M=/home/vboxuser/Desktop/src/os4 modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-49-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.2.0-23ubuntu4) 13.2.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
CC [M]  /home/vboxuser/Desktop/src/os4/morse_code.o
MODPOST /home/vboxuser/Desktop/src/os4/Module.symvers
LD [M]  /home/vboxuser/Desktop/src/os4/morse_code.ko
BTF [M] /home/vboxuser/Desktop/src/os4/morse_code.ko
skipping BTF generation for /home/vboxuser/Desktop/src/os4/morse_code.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-49-generic'
```

6. Нагрузочное тестирование драйвера

```
DEVICE="/dev/morse_code"

check_result() {
    local input="$1"
    local expected="$2"
    local result="$3"
    if [ "$result" == "$expected" ]; then
        echo "PASS: Input='$input' -> Output='$result'"
    else
        echo "FAIL: Input='$input' -> Output='$result' (Expected='$expected')"
    fi
}

if [ ! -e "$DEVICE" ]; then
    echo "Ошибка: Устройство $DEVICE не существует. Убедитесь, что драйвер
загружен." exit 1
fi

echo "Starting tests..."

declare -A tests=(
    ["0123456789"]="----- .---- .---- .---- .---- .---- .---- .---- .---- .----."
    ["HELLO WORLD"]=".... . .-.. .--- --- ..- -.- .- .-. -. ."
    ["CODE123"]="-.-. --- -.. . ---- .---- .---- ."
    ["hello"]=".... . .-.. .--- ---"
    ["A1B2C3"]=".- .---- -... .---- -... .----"
    ["WORLD"]=".-. --- .- .-. -. ."
    ["123"]=".---- .---- .----"
    ["TEST"]="-. . . -"
)

for input in "${!tests[@]}"; do
    expected="${tests[$input]}"

    echo -n "$input" > "$DEVICE"

    result=$(cat "$DEVICE")

    check_result "$input" "$expected" "$result"
done

echo "Tests completed."
```

```
PASS: Input='0123456789' -> Output='.....  
PASS: Input='HELLO WORLD' -> Output='...  
PASS: Input='CODE123' -> Output='...  
PASS: Input='hello' -> Output='...  
PASS: Input=' ' -> Output=' '  
PASS: Input='A1B2C3' -> Output='.-  
PASS: Input='WORLD' -> Output='.-  
PASS: Input='123' -> Output='...  
PASS: Input='TEST' -> Output='- . . . -'
```

7. Удаляем драйвер (как завершение цикла активности ПО):

```
sudo rmmod morse_code
```

Вывод:

В ходе выполнения работы была изучена разработка драйверов для операционной системы Linux, включая настройку окружения, написание драйвера, тестирование и отладку. Основной задачей было создание драйвера, преобразующего входные строки в код Морзе, и его проверка через тестовый bash-скрипт.